Computational Verification of Sun's Conjecture 4.3(i) and Exceptional Structure of Sequence A248044

Carlo Corti[1*]

[1*]Independent researcher, 21-040, Kalinówka, Poland.

Corresponding author(s). E-mail(s): carlo.corti@outlook.com;

Abstract

We present the first large-scale computational investigation of OEIS (On-Line Encyclopedia of Integer Sequences) sequence A248044, which encodes Sun's Conjecture 4.3(i): for every positive integer m, there exists a positive integer n such that $(m+n) \mid \pi(m)^2 + \pi(n)^2$, where $\pi(x)$ denotes the prime-counting function. We determine $a(m) = \min\{n \in \mathbb{Z}^+ \text{ (positive integers)} : (m+n) \mid \pi(m)^2 + \pi(n)^2\}$ for **99 998** of the first **100 000** positive integers, with a final search bound of $n \leq 10^{13}$. The computation employs a segmented sieve of Eratosthenes coupled with a modular obstruction filter based on quadratic residuacity of -1, parallelised via OpenMP on **16** threads. The two unresolved values, $m = 19\,623$ and $m = 19\,624$, satisfy $a(m) > 10^{13}$, yielding Cramér-type ratios $a(m)/m > 5.095 \times 10^8$ — the largest lower bounds in the entire sequence. We prove that the obstruction filter is sound (Obstruction Filter Lemma) and analyse the arithmetic structure of the unresolved cases: all belong to a cluster sharing $\pi(m) = 2225 = 5^2 \times 89$, whose prime factorisation consists entirely of primes congruent to $1 \pmod 4$, so that the implemented obstruction filter operates at full scope. Statistical analysis reveals a heavy-tailed distribution consistent with a log-exponential model, and we provide probabilistic estimates for the unresolved values.

Keywords: Prime counting function, Sum of two squares, Computational verification, OEIS A248044, Sun's conjecture, Obstruction filter

MSC Classification: 11A41 , 11Y55 , 11N05 , 11A07

1 Introduction

In a 2013 preprint subsequently published in 2019, Zhi-Wei Sun [1] proposed a collection of conjectures involving divisibility conditions on prime-related functions. Among these, Conjecture 4.3(i) (p. 15 of [1]) states:

Conjecture 1.1 (Sun, 2013/2019) *For every positive integer m, there exists a positive integer n such that*

$$(m+n) \mid \pi(m)^2 + \pi(n)^2, \tag{1.1}$$

where $\pi(x) = \#\{p \le x : p \text{ prime}\}$ denotes the prime-counting function.

The sequence $a(m)$, defined as the least positive integer n satisfying (1.1), was submitted to the OEIS in 2014 and appears as sequence A248044 [3]. Prior to this work, the OEIS b-file contained values only for $m \le 200$. To the best of our knowledge, no large-scale computational verification of Conjecture 1.1 has appeared in the literature (for a related computational study of a different Sun conjecture, see [15]).

The companion conjecture, 4.1(i), involving p_m (the m-th prime) in place of $\pi(m)$, was investigated computationally in [2]; the present paper extends the same methodology to the prime-counting variant.

The present work makes three main contributions.

First, we extend the computation of $a(m)$ to $m = 1, \ldots, 100\,000$, resolving 99 998 of 100 000 cases with a final search bound of $n \le 10^{13}$. This is the largest verified dataset for this conjecture.

Second, we carry out a systematic statistical analysis of the distribution of $a(m)$, revealing a heavy-tailed structure well described by a log-exponential model.

Third, we identify and characterise the two values of m for which the conjecture resists verification beyond $n = 10^{13}$, both belonging to an arithmetically structured cluster sharing a common value $\pi(m) = 2225 = 5^2 \times 89$; we explain how the factorisation of $\pi(m)$ into primes $\equiv 1 \pmod 4$ ensures that the implemented obstruction filter operates at full scope, and propose the anomalous resistance of these two cases as an open problem.

The paper is organised as follows. Section 2 reformulates the divisibility condition in modular arithmetic and establishes the theoretical basis for the obstruction filter. Section 3 describes the algorithm and implementation. Section 4 presents the computational results. Section 5 carries out the statistical analysis. Section 6 analyses the two unresolved cases. Section 7 concludes with open problems.

2 Mathematical Background

For positive integers m and n, set $s = m + n$. Throughout this section, $v_q(s)$ denotes the q-adic valuation of s, i.e., the largest exponent e such that $q^e \mid s$. The divisibility condition (1.1) is equivalent to

$$\pi(n)^2 \equiv -\pi(m)^2 \pmod{s}. \tag{2.1}$$

When $\gcd(\pi(m), s) = 1$, this simplifies to

$$\left(\pi(n) \cdot \pi(m)^{-1}\right)^2 \equiv -1 \pmod{s}, \tag{2.2}$$

so a solution exists at index n only if -1 is a quadratic residue modulo s (a necessary but not sufficient condition, since $\pi(n)$ must also fall in the correct residue class).

A classical result (see [5], Ch. 5) gives:

Proposition 2.1 *-1 is a quadratic residue modulo $s > 1$ if and only if both of the following hold:*

(a) *$4 \nmid s$ (equivalently, $v_2(s) \leq 1$, where $v_2(s)$ denotes the 2-adic valuation of s);*
(b) *no prime $q \equiv 3 \pmod 4$ divides s.*

Remark 2.2 Condition (b) is strictly stronger than requiring the Jacobi symbol $\left(\frac{-1}{s}\right) = 1$. For odd s, the Jacobi symbol equals 1 if and only if $\sum_{q \equiv 3\,(4)} v_q(s)$ is even; actual solvability of $x^2 \equiv -1 \pmod s$ requires that no such prime divides s at all. A standard example: $s = 21 = 3 \times 7$, where $\left(\frac{-1}{21}\right) = 1$ yet -1 is not a QR modulo 21.

2.1 The obstruction filter

We now formalise the obstruction that arises when a small prime $q \equiv 3 \pmod 4$ divides $s = m + n$ to an odd power but does not divide $\pi(m)$.

Lemma 2.3 (Obstruction Filter) *Let q be a prime with $q \equiv 3 \pmod 4$, and let $s = m + n$ with $m, n \in \mathbb{Z}^+$. Suppose that $q \mid s$ and that $q \nmid \pi(m)$. Then $s \nmid \pi(m)^2 + \pi(n)^2$.*

Proof Suppose for contradiction that $s \mid \pi(m)^2 + \pi(n)^2$. Since $q \mid s$, we have $q \mid \pi(m)^2 + \pi(n)^2$.

Since $q \nmid \pi(m)$ by hypothesis, we have $\gcd(\pi(m), q) = 1$, so $\pi(m)$ is invertible modulo q. From $q \mid \pi(m)^2 + \pi(n)^2$ we get

$$\left(\pi(n) \cdot \pi(m)^{-1}\right)^2 \equiv -1 \pmod{q}.$$

Since $q \equiv 3 \pmod 4$, the element -1 is not a quadratic residue modulo q ([5], Ch. 5, Theorem 3). This contradicts the equation above, so the assumption $s \mid \pi(m)^2 + \pi(n)^2$ must be false. □

Corollary 2.4 *If every prime factor of $\pi(m)$ is congruent to $1 \pmod 4$ (i.e., $\pi(m)$ has no prime factor $q \equiv 3 \pmod 4$), then for any prime $q \equiv 3 \pmod 4$ with $q \mid s$, the condition of Lemma 2.3 is satisfied and s is blocked. In this case the coprimality condition $q \nmid \pi(m)$ holds for every filter prime $q \in \{3, 7, 11, 19\}$, so the implemented filter operates at its full scope.*

Proof If every prime factor of $\pi(m)$ is $\equiv 1 \pmod 4$, then $q \nmid \pi(m)$ for every $q \equiv 3 \pmod 4$, and Lemma 2.3 applies whenever $q \mid s$. □

Remark 2.5 The condition $4 \mid s$ from Proposition 2.1(a) can be analysed via a direct parity argument, independent of the quadratic residue framework. If $4 \mid s$ and $s \mid \pi(m)^2+\pi(n)^2$, then $4 \mid \pi(m)^2 + \pi(n)^2$. Since $x^2 \bmod 4 \in \{0, 1\}$ for all integers x, the sum $\pi(m)^2 + \pi(n)^2 \bmod 4$ lies in $\{0, 1, 2\}$; it equals 0 if and only if both $\pi(m)$ and $\pi(n)$ are even. Therefore, when $\pi(m)$ is odd — as is the case for the unresolved values $m = 19\,623$ and $m = 19\,624$, where $\pi(m) = 2225$ — the condition $4 \mid s$ provably precludes a solution, regardless of whether $\gcd(\pi(m), s) = 1$. For m with $\pi(m)$ even, solutions with $4 \mid s$ are theoretically possible. Our implementation does not reject candidates based on $4 \mid s$; the analysis above is provided as theoretical context for the exceptional cases.

Remark 2.6 The proof of Lemma 2.3 is valid for any prime $q \equiv 3 \pmod 4$, not only for the small primes used in our implementation. Our code tests $q \in \{3, 7, 11, 19\}$ for computational efficiency; testing all primes $q \equiv 3 \pmod 4$ dividing s to odd power would make the odd-valuation filter exhaustive, but at prohibitive computational cost.

Remark 2.7 The implementation tests the condition "$v_q(s)$ is odd" rather than the weaker condition "$q \mid s$" stated in Lemma 2.3. Since $v_q(s)$ odd implies $q \mid s$, the implemented condition is strictly stronger: the code rejects a *subset* of the candidates that the Lemma permits rejecting. Candidates with even valuation $v_q(s) \geq 2$ are not filtered and proceed to the full divisibility test; no valid solution is ever missed. In other words, the implemented filter is *sound* but not maximal with respect to Lemma 2.3. The choice of the odd-valuation test was driven by computational efficiency: it enables a fast early-exit test (`if(s%q^2 != 0) return 1`) that avoids the full valuation loop in the common case $v_q(s) = 1$.

2.2 Density of obstructed candidates

Three progressively weaker obstruction conditions are relevant. *Condition* (b) of Proposition 2.1 requires that no prime $q \equiv 3 \pmod 4$ divides s at all. The classical B_1 class (integers representable as a sum of two squares) imposes the weaker requirement that every such prime appears to an *even* exponent; the set satisfying condition (b) is a proper subset of B_1. Finally, our implemented filter (Remark 2.7) tests only whether $v_q(s)$ is odd for a finite set of primes, rejecting a subset of the candidates that fail the B_1 condition.

The Landau–Ramanujan theorem [9] gives the count of B_1 integers up to x as $\sim Kx/\sqrt{\ln x}$ for a constant $K \approx 0.7642$; the asymptotic estimates rely on Mertens-type products [10, 11]. The count of integers satisfying the strictly stronger condition (b) has the same asymptotic order $\sim C_0 x/\sqrt{\ln x}$ but with a smaller constant [11]. At the finite scales relevant to our computation ($x \sim 10^9$ to 10^{13}), the fraction of candidates whose q-adic valuations are all even for every $q \equiv 3 \pmod 4$ (the B_1 condition) is roughly 17–20%; the fraction satisfying the full condition (b) is smaller still. In either case, 80% or more of candidates are obstructed in principle by odd-valuation primes.

Our implementation checks only $q \in \{3, 7, 11, 19\}$, which empirically eliminates approximately 57% of candidates (measured across the monolithic run for $m = 1, \ldots, 100\,000$), well below the theoretical maximum. The gap is due to the limited trial-division bound: candidates whose only factor $q \equiv 3 \pmod 4$ exceeds 19 are not eliminated.

3 Algorithm and Implementation

3.1 Overview

The computation proceeded in two phases: a *monolithic* phase computing $a(m)$ for all $m = 1, \ldots, 100\,000$ with search bound $n \le 6 \times 10^9$, followed by a series of *targeted segmented* searches extending the bound to $n = 10^{13}$ for the unresolved cases. Both phases use a sieve of Eratosthenes [6] for prime enumeration.

All computations were performed on a personal workstation: AMD Ryzen 9 7940HS (16 threads), 64 GB DDR5 RAM, running Linux Mint 22.3. Compilation used GCC with `-O3 -march=znver4 -fopenmp`.

3.2 Monolithic search: `sun_A248044_v5.c`

For the initial phase, the program builds a global table of $\pi(n)$ for all $n \le N_{\max}$, stored as `uint32_t`. The sieve of Eratosthenes marks composites in a `uint8_t` array; a prefix sum then fills the π-table. Memory consumption is $5 \cdot N_{\max}$ bytes (≈ 30 GB for $N_{\max} = 6 \times 10^9$).

For each m and each candidate n, the program sets $s = m + n$ and checks:

1. **Obstruction filter:** for each $q \in \{3, 7, 11, 19\}$, if $v_q(s)$ is odd and $q \nmid \pi(m)$, reject n (see Remark 2.7). Additionally, if s is a prime with $s \equiv 3 \pmod 4$ and $s > \pi(m)$ (which holds for all $m \ge 2$), reject n (since $v_s(s) = 1$ is odd and $s \nmid \pi(m)$, Lemma 2.3 applies).
2. **Divisibility test:** compute $\pi(m)^2 + \pi(n)^2$ and check whether s divides the sum.

For $n \le 6 \times 10^9$, we have $\pi(n) \le \pi(6 \times 10^9) \approx 2.80 \times 10^8$ and $\pi(m) \le \pi(100\,000) = 9\,592$, so $\pi(m)^2 + \pi(n)^2 < 7.82 \times 10^{16}$, which fits in `int64_t` (max $\approx 9.2 \times 10^{18}$).

The monolithic phase resolved 99 935 of 100 000 cases in approximately 10 minutes of wall-clock time, leaving 65 unresolved cases (the "exceptional" values of m).

3.3 Targeted segmented search: `sun_A248044_targeted_v3.c`

The monolithic architecture cannot scale beyond $\sim 7 \times 10^9$ due to the RAM ceiling of the global π-table. For the extended searches we developed a segmented approach: the range $[n_{\min}, n_{\max})$ is divided into segments of width $W = 10^9$, and for each segment $[l, l + W)$:

1. A segmented sieve of Eratosthenes marks composites in a `uint8_t` array `seg[]` of size W (≈ 1 GB).
2. A prefix sum computes $\pi_{\text{seg}}[k] = \#\{\text{primes in } [l, l+k]\}$ as `uint32_t` (≈ 4 GB), and the absolute count is recovered as $\pi(l + k) = \pi_{\text{base}} + \pi_{\text{seg}}[k]$, where $\pi_{\text{base}} = \pi(l)$ is maintained across segments.
3. The inner loop iterates over all $k \in [0, W)$ in parallel (OpenMP, `schedule(guided)`), and for each still-unresolved target m, applies the obstruction filter and divisibility test.

Total memory per segment: $\approx 5\,\text{GB}$. Since $\pi_{\text{seg}}[k]$ counts primes within a single segment of width 10^9, the offset cannot exceed $\sim 4 \times 10^7$ and comfortably fits in `uint32_t`.

For $n > \sim 3.5 \times 10^9$ we have $\pi(n) > 1.6 \times 10^8$, so $\pi(n)^2 > 2.6 \times 10^{16}$; and at the extremes of Ext-5 ($n \sim 10^{13}$) we have $\pi(n) \approx 3.5 \times 10^{11}$, giving $\pi(n)^2 \approx 1.2 \times 10^{23}$, which far exceeds the range of `int64_t`. Accordingly, the segmented tool uses `__int128` (a GCC extension) uniformly for the sum $\pi(m)^2 + \pi(n)^2$ and the modular reduction. On x86-64, GCC compiles `__int128 % int64_t` to a native `DIV` instruction (`rdx:rax`), so the overhead relative to 64-bit arithmetic is negligible (a few cycles per division).

3.4 Parallelism and correctness

The inner loop is parallelised with `#pragma omp parallel for schedule(guided)` [14]. The choice of `guided` scheduling reflects the non-uniform cost per iteration: the obstruction filter rejects $\sim 57\%$ of candidates without performing the expensive 128-bit division, creating significant load imbalance under static scheduling.

Target data (arrays of m, $\pi(m)^2$, and filter bitmasks) are copied into Structure-of-Arrays (SoA) layout before the parallel region. With at most 65 targets at ≤ 24 bytes each, the working set of $\approx 1.5\,\text{kB}$ fits entirely in the L1 cache (32 kB per core), ensuring that the hot loop is memory-bound on the segment arrays rather than on target metadata.

The variable `found_n[ai]` (recording the best solution found so far for target ai) is read without synchronisation inside the parallel loop and written only under `#pragma omp critical`. This constitutes a formal data race under the C11/OpenMP memory model; however, on x86-64 under the Total Store Order (TSO) memory model, an aligned 64-bit read cannot produce a torn value. The worst case is a stale read, which may cause one redundant iteration but never affects correctness: the critical section always compares and retains the global minimum. An `omp atomic read` directive would eliminate the formal race but at a cost of ~ 4 cycles $\times 6.5 \times 10^{10}$ iterations ≈ 87 seconds per segment — unacceptable given that the benign race preserves correctness on the target architecture. This correctness argument is specific to the x86-64 TSO memory model and would not hold on architectures with weaker memory ordering (e.g., ARM); however, all computations for this paper were performed on x86-64.

3.5 Code review

The targeted search code (v3) underwent two rounds of independent review with assistance from AI language models (see the AI Disclosure in Section 3.6). The first round identified two critical bugs (integer overflow in $\pi(n)^2$ and a race condition on the minimum-tracking logic); both were fixed in v2. The second round confirmed correctness and led to minor improvements (input parsing hardening, overflow fix in the CSV output column). All observations and their dispositions are documented in the source code header.

3.6 Use of AI tools

The computational implementation, including the C search programs and segmented sieve, was developed with extensive assistance from Anthropic's Claude language models (`claude-sonnet-4-6` and `claude-opus-4-6`). Claude generated substantial drafts of the algorithmic and statistical sections, performed two independent code reviews of the targeted search program, and assisted in the statistical analysis. All final text and numerical results were verified and approved by the author, who bears full scientific responsibility for all results, including those portions where AI assistance was used.

4 Computational Results

4.1 Campaign summary

Table 1 summarises the progressive resolution of the 65 initially unresolved cases across the monolithic and extended campaigns.

Table 1 Campaign summary. The monolithic run (M1) used `sun_A248044_v5.c`; all Ext runs used `sun_A248044_targeted_v3.c`. "+Resolved" denotes newly resolved cases in each run.

Run	Bound n	+Resolved	Still open	Time	Note
M1 (monolithic)	$\leq 6 \times 10^9$	99 935	65	∼10 min	All $m \leq 10^5$
Ext-2	$6 \times 10^9 \to 10^{10}$	36	29	137 s	
Ext-3	$10^{10} \to 10^{11}$	20	9	1 132 s	
Ext-4	$10^{11} \to 10^{12}$	4	5	9 452 s	
Ext-5	$10^{12} \to 10^{13}$	3	**2**	86 098 s	

The total computation time for the extended campaigns was approximately 96 819 seconds ($\approx$ 26.9 hours). Including the monolithic phase ($\approx$ 0.17 hours), the entire computation required approximately 27 hours of wall-clock time ($\approx$ 432 CPU-hours on 16 threads). An Ext-6 campaign ($10^{13} \to 10^{14}$) would require an estimated $\sim$ 240 hours of wall-clock time on the same hardware, assuming similar per-segment cost (actual time may vary with changing obstruction rates at higher bounds), and was deemed infeasible.

4.2 Largest values of $a(m)$

Table 2 lists the ten largest absolute values of $a(m)$ found in the computation.

4.3 Largest Cramér-type ratios

Table 3 lists the ten largest Cramér-type ratios $C(m) = a(m)/m$.

Table 2 Top 10 values of $a(m)$ by absolute magnitude.

m	$\pi(m)$	$a(m)$	Ratio $a(m)/m$	Campaign
83 115	8 117	4 629 736 117 663	5.57×10^7	Ext-5
40 963	4 289	2 650 225 478 854	6.47×10^7	Ext-5
19 620	2 225	1 150 315 774 321	5.86×10^7	Ext-5
57 919	5 867	330 283 186 011	5.70×10^6	Ext-4
57 920	5 867	330 283 186 010	5.70×10^6	Ext-4
45 151	4 687	229 736 032 374	5.09×10^6	Ext-4
93 381	9 019	190 261 921 669	2.04×10^6	Ext-4
85 504	8 321	79 705 931 373	9.32×10^5	Ext-3
44 857	4 661	32 348 326 705	7.21×10^5	Ext-3
44 858	4 661	32 348 326 704	7.21×10^5	Ext-3

Table 3 Top 10 values by Cramér-type ratio $a(m)/m$.

m	$\pi(m)$	$a(m)$	$a(m)/m$	Campaign
40 963	4 289	2 650 225 478 854	6.47×10^7	Ext-5
19 620	2 225	1 150 315 774 321	5.86×10^7	Ext-5
83 115	8 117	4 629 736 117 663	5.57×10^7	Ext-5
57 919	5 867	330 283 186 011	5.70×10^6	Ext-4
57 920	5 867	330 283 186 010	5.70×10^6	Ext-4
45 151	4 687	229 736 032 374	5.09×10^6	Ext-4
93 381	9 019	190 261 921 669	2.04×10^6	Ext-4
85 504	8 321	79 705 931 373	9.32×10^5	Ext-3
44 857	4 661	32 348 326 705	7.21×10^5	Ext-3
44 858	4 661	32 348 326 704	7.21×10^5	Ext-3

4.4 Distribution by order of magnitude

Table 4 shows the distribution of $a(m)$ across orders of magnitude, among the 99 998 resolved cases.

The monotonically decreasing counts across orders of magnitude confirm the heavy-tailed nature of the distribution. The median value is $a(m) = 52\,846$.

4.5 Independent verification of extended-campaign results

All 63 results from the extended campaigns (Ext-2 through Ext-5) were independently verified in two steps. First, the prime-counting values $\pi(a(m))$ reported in the CSV output files were recomputed using `primecount` 7.10 [17], an implementation of the Deleglise–Rivat algorithm that is entirely independent of the segmented sieve of Eratosthenes used in our search programs. In all 63 cases, the `primecount` output matched the CSV value exactly, confirming the correctness of the sieve-based $\pi(n)$

Table 4 Distribution of $a(m)$ by order of magnitude (99 998 resolved cases plus 2 unresolved).

Range	Count	Cumulative
$a(m) \le 10^6$	89 685	89 685
$10^6 < a(m) \le 10^7$	7 516	97 201
$10^7 < a(m) \le 10^8$	2 009	99 210
$10^8 < a(m) \le 10^9$	576	99 786
$10^9 < a(m) \le 10^{10}$	185	99 971
$10^{10} < a(m) \le 10^{11}$	20	99 991
$10^{11} < a(m) \le 10^{12}$	4	99 995
$10^{12} < a(m) \le 10^{13}$	3	99 998
$a(m) > 10^{13}$ (unresolved)	2	100 000

computation at the witness points. Second, the divisibility condition

$$(\pi(m)^2 + \pi(a(m))^2) \bmod (m + a(m)) = 0 \tag{4.1}$$

was verified directly using the `primecount`-confirmed values and Python's arbitrary-precision integers, with zero discrepancies. The full verification log is available in the project repository [16].

The values of $\pi(a(m))$ span from $\approx 2.8 \times 10^8$ (for $a(m) \approx 6 \times 10^9$, Ext-2) to $\approx 1.6 \times 10^{11}$ (for $a(m) \approx 4.6 \times 10^{12}$, Ext-5). As an additional cross-check, Table 5 lists standard checkpoints $\pi(10^k)$ from OEIS A006880 [4].

Table 5 Checkpoints for $\pi(n)$ used in independent verification (OEIS A006880, confirmed with SymPy 1.13 [13]).

n	$\pi(n)$
10^8	5 761 455
10^9	50 847 534
6×10^9	279 545 368
10^{10}	455 052 511
10^{11}	4 118 054 813
10^{12}	37 607 912 018
10^{13}	346 065 536 839

4.6 Consecutive-m pairs and the shadowing mechanism

A striking feature of the dataset is the prevalence of consecutive pairs $(m, m+1)$ sharing large values of a. The top-10 tables include the pairs $(57\,919, 57\,920)$ and $(44\,857, 44\,858)$, each with $a(m+1) = a(m) - 1$. The following proposition explains this phenomenon.

Proposition 4.1 (Shadowing) *Let $m \geq 1$ and suppose $a(m) > 1$. If $m+1$ is composite (i.e., $\pi(m+1) = \pi(m)$) and $a(m)$ is composite (i.e., $\pi(a(m)) = \pi(a(m)-1)$), then $a(m+1) \leq a(m) - 1$.*

Proof Set $n^* = a(m) - 1$. Then $n^* \geq 1$ since $a(m) > 1$. Since $m+1$ is composite, $\pi(m+1) = \pi(m)$. Since $a(m)$ is composite, $\pi(n^*) = \pi(a(m) - 1) = \pi(a(m))$. The new modulus is

$$s' = (m+1) + n^* = (m+1) + (a(m) - 1) = m + a(m) = s,$$

where $s = m + a(m)$ is the modulus from the solution for m. Therefore

$$\pi(m+1)^2 + \pi(n^*)^2 = \pi(m)^2 + \pi(a(m))^2,$$

which is divisible by $s = s'$ by the definition of $a(m)$. Hence n^* is a valid witness for $m+1$, giving $a(m+1) \leq n^* = a(m) - 1$. □

Remark 4.2 The shadowing mechanism is not guaranteed: either $m+1$ or $a(m)$ could be prime, in which case the hypotheses of Proposition 4.1 fail. When both conditions hold, the proposition produces exact pairs with $a(m+1) = a(m) - 1$ provided $n^* = a(m) - 1$ is indeed the minimum. Empirically, equality $a(m+1) = a(m) - 1$ holds in all observed instances, suggesting that when the shadowing mechanism applies, $n^* = a(m) - 1$ is typically the first valid witness for $m+1$.

5 Statistical Analysis

5.1 Heavy-tailed distribution

The distribution of $a(m)$ is strongly heavy-tailed (cf. [12] for the Hill estimator framework): while the median is approximately 52 846, the maximum resolved value is $\approx 4.63 \times 10^{12}$, a ratio of $\sim 8.8 \times 10^7$. Values of $a(m)$ exceeding 10^9 occur for 212 values of m out of 99 998 resolved (0.212%), while values exceeding 10^{12} occur for 3 values (0.003%).

5.2 A log-exponential model

We model the occurrence of solutions heuristically. For a given m, each candidate $s = m + n$ (not blocked by the obstruction filter) independently has a "pass probability" that depends on the number of valid residue classes for $\pi(n)$ modulo s. Under a Cramér-type heuristic [7, 8], the number of solutions in an interval $[N_1, N_2]$ is approximately proportional to $\ln(N_2/N_1)$, giving a log-exponential distribution:

$$\Pr\big(a(m) < N \mid a(m) > N_0\big) = 1 - \exp\big(-\lambda \cdot \ln(N/N_0)\big) = 1 - (N_0/N)^\lambda, \qquad (5.1)$$

where $\lambda > 0$ is a pass-rate parameter.

5.3 Calibration from extended campaigns

The pass-rate parameter λ can be estimated from the extended campaigns by the maximum-likelihood relation

$$\hat{\lambda} = \frac{-\ln(1 - r/t)}{\ln(N_{\max}/N_{\min})}, \tag{5.2}$$

where r and t denote the number of cases resolved and the number of targets entering the campaign. (The first-order Taylor approximation $\lambda \approx r/(t \ln(N_{\max}/N_{\min}))$ underestimates $\hat{\lambda}$ when r/t is large.) Table 6 summarises the estimates.

Table 6 Empirical pass rates by campaign, estimated via Equation (5.2). The declining trend reflects the selection effect: cases surviving to higher bounds have intrinsically lower pass rates.

Campaign	Resolved/Targets	$\ln(N_{\max}/N_{\min})$	$\hat{\lambda}$
Ext-2	36/65	0.511	1.58
Ext-3	20/29	2.303	0.51
Ext-4	4/9	2.303	0.26
Ext-5	3/5	2.303	0.40

The declining pass rate from Ext-2 to the later campaigns reflects a strong selection effect: the cases surviving to higher bounds are precisely those with the highest intrinsic obstruction, so the average pass rate among survivors decreases with the bound.

Using the Ext-5 estimate $\hat{\lambda} \approx 0.40$ for the two unresolved cases, the conditional median (given $a(m) > 10^{13}$) is

$$\tilde{N} = 10^{13} \cdot \exp(\ln 2/0.40) \approx 10^{13} \times 5.7 \approx 5.7 \times 10^{13}.$$

5.4 Probabilistic estimates for unresolved cases

Under the log-exponential model with $\hat{\lambda} = 0.40$, the *heuristic* probability of resolving each unresolved case within various bounds is:

$$\begin{aligned}
\Pr\big(a(m) \le 10^{14} \mid a(m) > 10^{13}\big) &\approx 1 - 10^{-0.40} \approx 0.60, \\
\Pr\big(a(m) \le 10^{15} \mid a(m) > 10^{13}\big) &\approx 1 - 10^{-0.80} \approx 0.84, \\
\Pr\big(a(m) \le 10^{16} \mid a(m) > 10^{13}\big) &\approx 1 - 10^{-1.19} \approx 0.94.
\end{aligned}$$

The heuristic probability of finding *both* cases within $[10^{13}, 10^{14}]$ is approximately $0.60^2 \approx 0.36$, and within $[10^{13}, 10^{15}]$ approximately $0.84^2 \approx 0.71$.

Remark 5.1 The log-exponential model treats $a(m)$ as though each logarithmic unit of search space is equally likely to yield a solution. This is a heuristic, not a theorem; the data are

deterministic, not i.i.d., and the model does not account for the specific arithmetic structure of individual cases. The estimates above should be understood as order-of-magnitude guidance, not precise probabilities.

6 Exceptionally Resistant Cases

6.1 The two unresolved values

After the full campaign (bound $n = 10^{13}$), the two values

$$m = 19\,623 \quad \text{and} \quad m = 19\,624 \tag{6.1}$$

remain unresolved, with

$$a(19\,623) > 10^{13}, \qquad a(19\,624) > 10^{13},$$

giving Cramér-type lower bounds $a(m)/m > 5.095 \times 10^8$ — the largest in the entire dataset.

The exhaustive search establishes the following.

Theorem 6.1 (Computational lower bounds) *For each $m \in \{19\,623,\ 19\,624\}$, the divisibility condition $(m+n) \mid \pi(m)^2 + \pi(n)^2$ has no solution with $1 \le n \le 10^{13}$. Consequently, either $a(m) > 10^{13}$ or Conjecture 1.1 fails for this value of m.*

Proof The full range $1 \le n \le 10^{13}$ was covered in two phases. The monolithic program `sun_A248044_v5.c` (Section 3) tested every n in the range $1 \le n \le 6 \times 10^9$ for all $m \le 100\,000$, finding no solution for the two values in (6.1). The targeted segmented program `sun_A248044_targeted_v3.c` (Section 3.3) then extended the search over the intervals $6 \times 10^9 < n \le 10^{13}$ via the campaigns Ext-2 through Ext-5 (Table 1), again finding no solution for these two values. Both programs enumerate values of $\pi(n)$ via a sieve of Eratosthenes and, for each n not eliminated by the modular filter of Section 2, evaluate the divisibility condition $(\pi(m)^2 + \pi(n)^2) \bmod (m+n) = 0$ directly (using `int64_t` in the monolithic phase and `__int128` in the segmented phase). The modular filter is *sound* (Lemma 2.3), including the self-blocking prime check, which is a special case of the Lemma with $q = s$: it eliminates only values of n for which no solution can exist, so no valid candidate is skipped. For both values of m, the search returns no solution in the stated range; the sieve is deterministic and complete, so the conclusion follows. All 63 resolved extended-search cases were independently verified: the prime-counting values $\pi(a(m))$ were recomputed with `primecount` [17] and the divisibility condition (4.1) was confirmed with zero discrepancies (Section 4). The source code is publicly available [16]. □

Remark 6.2 The result of Theorem 6.1 is conditional on the correctness of the deterministic search implementation. All 63 resolved extended-search cases have been independently verified: the values $\pi(a(m))$ were recomputed using `primecount` [17] (Deleglise–Rivat algorithm, independent of our sieve), all 63 values matched exactly, and the divisibility condition (4.1)

was confirmed with zero discrepancies. This includes the three resolved cases with the largest absolute values (Table 2).

6.2 The cluster $\{19\,620, 19\,623, 19\,624\}$

The three values $m \in \{19\,620, 19\,623, 19\,624\}$ all satisfy $\pi(m) = 2225 = 5^2 \times 89$. The value $m = 19\,620$ was resolved in Ext-5 at $a(19\,620) = 1\,150\,315\,774\,321$ (ratio $\approx 5.86 \times 10^7$), but $m = 19\,623$ and $m = 19\,624$ resist beyond 10^{13} — a factor of at least approximately 8.7 further.

The factorisation $2225 = 5^2 \times 89$ consists entirely of primes congruent to 1 (mod 4) ($5 \equiv 1 \pmod 4$ and $89 \equiv 1 \pmod 4$). By Corollary 2.4, the implemented obstruction filter operates at its full scope for all three values, since no filter prime $q \in \{3, 7, 11, 19\}$ (all $\equiv 3 \pmod 4$) divides $\pi(m) = 2225$.

This full-scope obstruction explains why all three values are exceptional, but it does not explain why $m = 19\,623$ and $m = 19\,624$ resist so much more than $m = 19\,620$.

6.3 Congruence analysis

Table 7 compares the residues of the three cluster members modulo small numbers.

Table 7 Residues of the cluster $\{19\,620, 19\,623, 19\,624\}$ modulo small values.

m	m mod 3	m mod 4	m mod 7	m mod 11	m mod 19	Status
19 620	0	0	6	7	12	Resolved (1.15×10^{12})
19 623	0	**3**	2	10	15	Open ($> 10^{13}$)
19 624	1	0	3	**0**	16	Open ($> 10^{13}$)

Notable features:

- $m = 19\,623 \equiv 3 \pmod 4$: for candidate $n \equiv 2 \pmod 4$, $s = m + n \equiv 1 \pmod 4$ (favourable); but for $n \equiv 1 \pmod 4$, $s \equiv 0 \pmod 4$ (blocked by condition (a)). This creates a different interaction pattern with the other obstruction filters than for the other two cluster members.
- $11 \mid 19\,624$: since $11 \equiv 3 \pmod 4$ and $11 \nmid \pi(19\,624) = 2225$, any s with $v_{11}(s)$ odd is blocked. The constraint $11 \mid m$ forces $s = m + n \equiv n \pmod{11}$, creating a periodic pattern that may interact unfavourably with the other obstructions.

The precise mechanism by which these congruence patterns produce an extra factor of at least approximately 8.7 in resistance relative to $m = 19\,620$ remains unexplained, and we propose it as an open problem (see Section 7).

7 Conclusions and Open Problems

We have determined $a(m)$ for 99 998 of the first 100 000 positive integers, with final search bound $n \leq 10^{13}$, providing, to the best of our knowledge, the first large-scale

computational verification of Sun's Conjecture 4.3(i). The dataset reveals a heavy-tailed distribution well described by a log-exponential model, with record values reaching $a(83\,115) = 4\,629\,736\,117\,663$ and Cramér-type ratios up to 6.47×10^7.

The progressive resolution of the exceptional cases — from 65 unresolved at bound 6×10^9, to 29 at 10^{10}, to 9 at 10^{11}, to 5 at 10^{12}, and finally to 2 at 10^{13} — strongly supports the conjecture: the pattern is consistent with a heavy-tailed distribution where every case is eventually resolved at a sufficiently high bound, rather than with the existence of structural counterexamples.

Comparison with Conjecture 4.1(i).

A companion computation [2] applies the same methodology to sequence A247975, which encodes Sun's Conjecture 4.1(i): $(m + n) \mid p_m^2 + p_n^2$. Computing $a(m)$ for $m \leq 120\,000$ with bound $n \leq 2 \times 10^{11}$ resolves 119 995 of 120 000 cases, leaving 5 unresolved. The comparison reveals instructive structural differences: the obstruction rate is higher (73–77% vs. $\sim$57% here), the maximum resolved Cramér-type ratio is smaller ($\sim 1.97 \times 10^5$ vs. $\sim 6.5 \times 10^7$), and no arithmetically structured cluster analogous to $\{19\,620, 19\,623, 19\,624\}$ appears among the unresolved cases of A247975. These contrasts indicate that the difficulty of Sun's divisibility conjectures depends critically on the arithmetic function involved (p_k vs. $\pi(k)$), not only on the shared divisibility framework $(m + n) \mid f(m)^2 + f(n)^2$.

We conclude with three open problems.

Open Problem 7.1 *Determine $a(19\,623)$ and $a(19\,624)$. Both satisfy $a(m) > 10^{13}$, with $a(m)/m > 5.095 \times 10^8$. The heuristic model suggests a median around 5.7×10^{13}, with approximately 84% probability of resolution within $n \leq 10^{15}$. An Ext-6 campaign ($10^{13} \to 10^{14}$) would require $\sim$ 240 hours on our hardware; algorithmic improvements (e.g., bitset sieve with 4× segment width, extended filter with $q < 200$) might reduce this to $\sim$ 35–40 hours.*

Open Problem 7.2 *Explain the anomalous resistance of $m = 19\,623$ and $m = 19\,624$ relative to $m = 19\,620$, all sharing $\pi(m) = 2225$. The resolved case $a(19\,620) \approx 1.15 \times 10^{12}$ is already extreme, yet the unresolved cases exceed it by a factor of at least approximately 8.7. Is there an arithmetic characterisation of the congruence conditions on m (beyond the factorisation of $\pi(m)$) that predicts this extra resistance?*

Open Problem 7.3 *Develop a refined probabilistic model for $a(m)$ that accounts for the dependence on the factorisation of $\pi(m)$. The current log-exponential model treats all unresolved cases uniformly; a model incorporating the obstruction rate (which depends on the prime factorisation of $\pi(m)$ and the congruence class of m) would provide sharper predictions.*

Acknowledgements. The author thanks Zhi-Wei Sun for the beautiful conjecture that motivated this work. The use of AI tools in the development of this work is disclosed in Section 3.6.

Declarations

Funding No funding was received for conducting this study.
Competing interests The author has no relevant financial or non-financial interests to disclose.
Data availability The complete dataset (99 998 resolved values and the 2 unresolved m-values), the full CSV outputs for all campaigns, and machine-readable verification logs are publicly available at:

- GitHub repository: https://github.com/carcorti/A248044
- Permanent Zenodo archive (DOI): https://doi.org/10.5281/zenodo.19182594
- OEIS: https://oeis.org/A248044

Code availability The C source code for the monolithic search (`sun_A248044_v5.c`), the segmented targeted search (`sun_A248044_targeted_v3.c`), and a Python verification script (`verify_A248044.py`) that checks the divisibility condition $(m + a(m)) \mid \pi(m)^2 + \pi(a(m))^2$ for any supplied list of $(m, a(m))$ pairs are available at the GitHub and Zenodo repositories listed above.
Author contribution Not applicable (single author).
AI assistance The author used Claude (Anthropic) as an AI assistant during the preparation of this manuscript. The AI assisted with code review, verification of computational results, and editorial drafting of the manuscript. All scientific content, mathematical proofs, computational results, and final decisions regarding the text are the sole responsibility of the author.

References

[1] Sun, Z.-W.: Some new problems in additive combinatorics. Nanjing Univ. J. Math. Biquarterly **36**(2), 134–155 (2019). Preprint: arXiv:1309.1679 [math.NT] (submitted 2013, v9: 2020). Conjecture 4.3(i) appears on p. 15. https://arxiv.org/abs/1309.1679

[2] Corti, C.: Exceptional cases and statistical structure of sequence A247975: a computational study of Sun's Conjecture 4.1(i). Preprint, 2026; data and code: Zenodo https://doi.org/10.5281/zenodo.18920371; OEIS b-file extension published March 2026, https://oeis.org/A247975

[3] OEIS Foundation: Sequence A248044. The On-Line Encyclopedia of Integer Sequences (accessed March 2026). https://oeis.org/A248044

[4] OEIS Foundation: Sequence A006880 ($\pi(10^n)$). The On-Line Encyclopedia of Integer Sequences (accessed March 2026). https://oeis.org/A006880

[5] Ireland, K., Rosen, M.: A Classical Introduction to Modern Number Theory, 2nd edn. Springer, New York (1990)

[6] Crandall, R., Pomerance, C.: Prime Numbers: A Computational Perspective, 2nd edn. Springer, New York (2005)

[7] Cramér, H.: On the order of magnitude of the difference between consecutive prime numbers. Acta Arith. **2**, 23–46 (1936)

[8] Granville, A.: Harald Cramér and the distribution of prime numbers. Scand. Actuarial J. **1995**(1), 12–28 (1995)

[9] Landau, E.: Über die Einteilung der positiven ganzen Zahlen in vier Klassen nach der Mindestzahl der zu ihrer additiven Zusammensetzung erforderlichen Quadrate. Arch. Math. Phys. **13**, 305–312 (1908)

[10] Mertens, F.: Ein Beitrag zur analytischen Zahlentheorie. J. Reine Angew. Math. **78**, 46–62 (1874)

[11] Tenenbaum, G.: Introduction to Analytic and Probabilistic Number Theory. Cambridge Univ. Press (1995)

[12] Hill, B.M.: A simple general approach to inference about the tail of a distribution. Ann. Statist. **3**, 1163–1174 (1975). https://doi.org/10.1214/aos/1176343247

[13] Meurer, A., et al.: SymPy: symbolic computing in Python. PeerJ Computer Science **3**, e103 (2017). https://doi.org/10.7717/peerj-cs.103

[14] OpenMP Architecture Review Board: OpenMP Application Programming Interface, Version 5.2 (2021). https://www.openmp.org/specifications/

[15] Machiavelo, A., Tsopanidis, A.: Zhi-Wei Sun's 1-3-5 conjecture and variations. J. Number Theory **222**, 135–155 (2021). https://doi.org/10.1016/j.jnt.2020.10.001

[16] Corti, C.: A248044: computational verification of Sun's Conjecture 4.3(i). GitHub repository and Zenodo archive (2026). https://github.com/carcorti/A248044. https://doi.org/10.5281/zenodo.19182594

[17] Walisch, K.: primecount: a high-performance prime counting implementation (Deleglise–Rivat algorithm), version 7.10 (2024). https://github.com/kimwalisch/primecount